Assignment 5: Optimising Attention on a CPU

CS6886: Systems for Deep Learning, IIT Madras

Yukash Ram CS23B069
https://github.com/Yukash09/CS6886_Assignment_5

Q1. Your machine **(20 marks)**

Fill in `machine_spec.txt`: CPU model, cores (physical, logical, performance and efficiency), clock frequencies, SIMD width and FMA units, cache sizes, memory type and size. From these, compute the peak FP32 GFLOP/s (one core and all cores), the memory bandwidth and the ridge point. Show your calculations in the report.

Machine spec

CPU model	AMD Ryzen 5 7520U with Radeon Graphics
Microarchitecture	Zen 2 (Mendocino)
Physical cores	4
Logical cores (hardware threads)	8
Performance / efficiency cores	N/A
Base clock	2.8 GHz
Max single-core clock	4.3 GHz
All-core clock under load	4.07 GHz
SIMD instruction set and width	AVX2 + FMA3, 256-bit (8× FP32)
FMA units per core	1 (256-bit)
Vector registers per core	16 (YMM, 256-bit)
L1 data cache (per core)	32 KiB
L2 cache (size, shared by)	512 KiB, per core
L3 cache (size, shared by)	4 MiB, all 4 cores
Cache line size	64 B
RAM size	16 GB
RAM type, speed, channels	LPDDR5, 5500 MT/s, 2 channels (2× 32-bit)
Operating system	Fedora Linux 43, kernel 7.2.13
Compiler and version	clang 21.1.8, `-O3 -march=native`
Peak FP32, one core	68.8 GFLOP/s
Peak FP32, all cores	260.5 GFLOP/s
Memory bandwidth	8.06 GB/s (1 thread), 15.5 GB/s (4 threads)
Ridge point, one core	8.5 FLOP/B
Ridge point, all cores	16.8 FLOP/B

Peak FP32 GFLOP/s

$$P = \text{cores} \times f \times \text{SIMD lanes} \times \text{FMA units} \times 2\ \tfrac{\text{FLOP}}{\text{FMA}}$$

where f is the clock, SIMD lanes the FP32 values per vector register, and FMA units the vector FMAs a core can start per cycle. P_1 is for one core, P_4 for all four.

- SIMD lanes = 8: a 256-bit AVX2 register holds eight 32-bit floats. The flags in `/proc/cpuinfo` list `avx2` and `fma` but no `avx512f`.
- FMA units = 1, measured: a loop of 12 independent FMAs, pinned to one core, runs at 1.00 FMA/cycle for 256- and 128-bit vectors, and 6 FMAs mixed with 6 adds at 2.00/cycle (`perf stat -e cycles:u taskset -c 2 ./fma256`). Zen 2 is documented with 2.[1]
- $f = 4.3\,\text{GHz}$ for one core (rated; 4.29 GHz observed) and 4.07 GHz for all cores under load:

  ```
  $ cat /sys/devices/system/cpu/cpu*/cpufreq/scaling_cur_freq
  ```

- cores = 4, since the two SMT threads of a core share its FMA unit.

$$P_1 = 1 \times 4.3 \times 8 \times 1 \times 2 = 68.8 \text{ GFLOP/s}, \qquad P_4 = 4 \times 4.07 \times 8 \times 1 \times 2 = 260.5 \text{ GFLOP/s}$$

Memory bandwidth

$$B_{\text{theory}} = \text{transfer rate} \times \text{bus width}, \qquad B_{\text{measured}} = \text{STREAM Triad}$$

where B_1 and B_4 are measured with 1 and 4 threads.

- 5500 MT/s and a 2×32-bit (8 B) bus (`sudo dmidecode -t memory`).
- STREAM Triad with 10^7-element arrays (76 MiB each, larger than L3):

  ```
  $ OMP_NUM_THREADS=1 ./stream_omp   # and OMP_NUM_THREADS=4
  ```

$$B_{\text{theory}} = 5500 \times 10^6 \times 8 = 44.0 \text{ GB/s}, \qquad B_1 = 8.06 \text{ GB/s}, \qquad B_4 = 15.5 \text{ GB/s}$$

Ridge point

$$I^* = \frac{P}{B}$$

where I^* is the arithmetic intensity (FLOP per DRAM byte) at which the memory roof $I \cdot B$ meets the compute roof P: below it a kernel is memory-bound, above it compute-bound.

$$I_1^* = \frac{68.8}{8.06} = 8.5 \text{ FLOP/B}, \qquad I_4^* = \frac{260.5}{15.5} = 16.8 \text{ FLOP/B}$$

[1] A. Fog, *The microarchitecture of Intel, AMD, and VIA CPUs*, §22, Table 22.1 and p. 234 (2026-05-23), `https://www.agner.org/optimize/microarchitecture.pdf`.

Q2. Naive baseline **(20 marks)**

Run `make naive`. For each input, report the time, GFLOP/s and percentage of the peak. How does the time change with N and with d? Explain where the time goes.

Time, GFLOP/s and percentage of peak

$$\text{GFLOP/s} = \frac{4N^2 d}{t}, \qquad \%\text{ of peak} = \frac{\text{GFLOP/s}}{P_1} \times 100$$

where t is the time of one `attention()` call (median of three `make naive` runs, each the driver's median of 5), $4N^2 d$ the driver's FLOP count (both matmuls; softmax not counted) and $P_1 = 68.8\,\text{GFLOP/s}$.

Input	Time (ms)	GFLOP/s	% of P_1	$QK^\top$ (ms)	softmax (ms)	PV (ms)
$N = 1024,\ d = 64$	133.3	2.01	2.92	46.7	3.9	78.9
$N = 2048,\ d = 64$	535.4	2.01	2.92	193.9	16.4	319.2
$N = 4096,\ d = 64$	2211.9	1.94	2.82	799.7	65.2	1306.8
$N = 4096,\ d = 128$	5528.0	1.55	2.25	2150.2	64.9	3216.4

Scaling

$$\frac{t'}{t} = \left(\frac{N'}{N}\right)^2 \frac{d'}{d}$$

where t, t' are the times at (N, d) and (N', d'), assuming a constant cost per FLOP.

Change	Expected	Measured
N: 1024 $\to$ 2048 ($d = 64$)	$\times 4$	$\times 4.02$
N: 2048 $\to$ 4096 ($d = 64$)	$\times 4$	$\times 4.13$
d: 64 $\to$ 128 ($N = 4096$)	$\times 2$	$\times 2.50$

Time breakdown

$$\text{FLOP/cycle} \le \frac{\text{FLOP per FMA}}{\text{FMA latency}} = \frac{2}{5} = 0.4$$

for a chain of dependent scalar FMAs (latency 5 cycles, Agner Fog p. 234[1]).

To see where the time goes, each step was timed separately in a copy of `attn_naive.c` (`now()` around each step, median of 5). The table above shows that PV takes 59–61% of the time and $QK^\top$ 36–40%; the softmax is only 1–3%. So the two matmuls are the problem:

```
/* 1. S = Q K^T * scale */
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++) {
        float acc = 0.0f;
        for (int k = 0; k < d; k++)
            acc += Q[(size_t)i * d + k] * K[(size_t)j * d + k];
        S[(size_t)i * N + j] = acc * scale;
    }
```

```
/* 3. O = S V */
for (int i = 0; i < N; i++)
    for (int k = 0; k < d; k++) {
        float acc = 0.0f;
        for (int j = 0; j < N; j++)
            acc += S[(size_t)i * N + j] * V[(size_t)j * d + k];
        O[(size_t)i * d + k] = acc;
    }
```

In both, the innermost loop adds into a single `acc`. Vectorising that would change the order of the floating-point additions, which clang won't do without fast-math:

```
$ clang -O3 -march=native -Rpass-missed=loop-vectorize -Rpass-analysis=loop-vectorize -c
    src/attn_naive.c
```

prints *cannot prove it is safe to reorder floating-point operations* for both loops, and in `objdump -d build/naive` the hot loop is a single scalar `vfmadd231ss`. Every FMA waits for the previous one, so the bound above applies, and the measured rate is close to it:

```
$ perf stat -e cycles:u,instructions:u ./build/naive input/qkv_N1024_d64.bin /dev/null
```

counts 3.30×10^9 cycles for the 6 calls of a run, so

$$\frac{6 \times 4N^2 d}{\text{cycles}} = \frac{1.61 \times 10^9}{3.30 \times 10^9} = 0.49 \text{ FLOP/cycle},$$

against a peak of 16.

PV takes longer than $QK^\top$ because of its access pattern. Its inner loop walks down a column of V, a stride of d floats (256 B), so only 4 bytes of each 64-byte line are used. $QK^\top$ walks along a row of K instead.

Doubling d costs $\times 2.50$ rather than $\times 2$ because both matmuls slow down more than expected: $QK^\top$ by $\times 2.69$ and PV by $\times 2.46$. The two have different causes. For $QK^\top$ it does not depend on the data size: on an extra random input with $N = 1024$, $d = 128$ it runs at 2.08 GFLOP/s, about the same as at $N = 4096$ (2.00), so it comes with the longer chain of 128 FMAs per output. For PV it does depend on size. At $N = 1024$ doubling d only costs $\times 2.10$ (78.3 to 164.8 ms), while at $N = 4096$ the L2 miss rate goes from 53% to 74%, with 1.9× as many misses per FLOP:

```
$ perf stat -e l2_cache_req_stat.ic_dc_hit_in_l2:u,l2_cache_req_stat.ic_dc_miss_in_l2:u
    ./build/naive input/qkv_N4096_d64.bin /dev/null   # and d128
```

Q3. Optimisations **(20 marks)**

Write each variant, run it, and report the same measurements as Q2 plus the speedup over the naive variant. `make check` must pass. Explain what each change does and what limits performance afterwards. Do not use BLAS or other math libraries.

Baseline

$$\text{speedup} = \frac{t_{\texttt{naive}}}{t}$$

Time, GFLOP/s and % of peak are measured as in Q2, and per-step times again come from instrumented copies. The page-fault and L2-miss counts below are referred to in several parts; they are for one run (6 calls) at $N = 4096$, $d = 64$:

```
$ perf stat -e page-faults:u,l2_cache_req_stat.ic_dc_miss_in_l2:u ./build/<variant>
    input/qkv_N4096_d64.bin /dev/null
```

Variant	Page faults	L2 misses
`loops`	99,158	74.0×10^6
`tiled`	99,157	126.2×10^6
`fused`	3,282	114.4×10^6
`best`	861	3.7×10^6

(a) `attn_loops.c`

`attn_loops.c`: reorder the loops so the compiler can vectorise them.

Implementation

The fix is to take the summed index out of the innermost loop. Then each iteration updates a different output, and the compiler can vectorise across outputs without reordering any sum. For PV this just means the order $i \to j \to k$. For $QK^\top$ the same idea would read K down a column, so K is first transposed into `Kt` and the order becomes $i \to k \to j$:

```
for (int i = 0; i < N; i++) {
    float *Si = S + (size_t)i * N;
    for (int j = 0; j < N; j++){
         Si[j] = 0.0f;
    }

    for (int k = 0; k < d; k++) {
        const float q = Q[(size_t)i * d + k];
        const float *Ktk = Kt + (size_t)k * N;
        for (int j = 0; j < N; j++){
            Si[j] += q * Ktk[j];
        }
    }
    for (int j = 0; j < N; j++){
        Si[j] *= scale;
    }
}
```

```
for (int i = 0; i < N; i++) {
    float *Oi = O + (size_t)i * d;
    for (int k = 0; k < d; k++){
        Oi[k] = 0.0f;
    }

    for (int j = 0; j < N; j++) {
        const float s = S[(size_t)i * N + j];
        const float *Vj = V + (size_t)j * d;
        for (int k = 0; k < d; k++){
            Oi[k] += s * Vj[k];
        }
    }
}
```

Both inner loops now vectorise; clang reports *vectorized loop (vectorization width: 8, interleaved count: 4)* for each:

```
$ clang -O3 -march=native -Rpass=loop-vectorize -Rpass-missed=loop-vectorize -c src/
    attn_loops.c
```

Input	Time (ms)	GFLOP/s	% of P_1	Speedup
$N = 1024,\ d = 64$	14.9	17.96	26.1	×8.92
$N = 2048,\ d = 64$	59.4	18.08	26.3	×9.02
$N = 4096,\ d = 64$	269.2	15.96	23.2	×8.22
$N = 4096,\ d = 128$	427.9	20.08	29.2	×12.92

Limitations

The running sums (`Si[j]`, `Oi[k]`) are now in memory. Each FMA has to load the old value and store the new one, on top of loading its input: 2 loads and 1 store, which is the most a Zen 2 core can do in a cycle (Agner Fog, p. 234[1]). The compiled loop in `objdump -d build/loops` is exactly that: `vmovups`, `vfmadd213ps` with a memory operand, `vmovups` store. At $N = 1024$ the matmuls reach 18 GFLOP/s ($QK^\top$, 7.3 ms) and 17 GFLOP/s (PV, 8.0 ms). Moving the sums into registers in (b) takes them to 49 and 57 GFLOP/s.

The softmax was not touched and is still scalar; clang reports its max and `expf` loops (lines 57 and 61) as *not vectorized*. At $N = 4096$, $d = 64$ the steps take 1.5 ms (transpose), 107.3 ms ($QK^\top$), 63.9 ms (softmax) and 80.6 ms (PV), so softmax has grown to 25% of the 253 ms.

(b) `attn_tiled.c`

`attn_tiled.c`: cache and register tiling.

Implementation

Register tiling. Each matmul works on an 8×8 block of outputs held in a small local array. clang keeps the array in registers, and every row of `Kt` that is loaded is used by 8 FMAs:

```
for(int i0 = 0 ; i0 < N ; i0 += IB){
    for(int j = 0 ; j < N ; j += C){
```

```
        for(int i = i0 ; i < i0 + IB ; i += R){
            float acc[R][C] = {{0}} ;
            for(int k = 0 ; k < d ; k++){
                for(int r = 0 ; r < R ; r++){
                    float q = Q[(size_t)(i+r)*d + k] ;
                    for(int c = 0 ; c < C ; c++){
                        acc[r][c] += q*Kt[(size_t)k*(N+16) + j + c] ;
                    }
                }
            }
            for(int r = 0 ; r < R ; r++){
                for(int c = 0 ; c < C ; c++){
                    S[(size_t)(i+r)*N + j + c] = acc[r][c] * scale ;
                }
            }
        }
    }
}
```

In `objdump -d build/tiled` the $QK^\top$ inner loop is one load, 8 broadcasts and 8 `vfmadd231ps` into `ymm1`–`ymm8`; there is no store until the block is done.

Cache tiling. Rows are processed `IB` at a time, so the slice of `Kt` (or V) loaded for one tile is still in cache when the next tile in the block needs it. To pick `IB`, a copy of `attn_tiled.c` was built with `-DIB=`16 to 256 and run twice each (ms):

`IB`	16	32	64	128	256
$N = 4096,\ d = 64$	184.0–185.6	190.9–191.5	187.4–188.4	187.3–188.7	196.9–197.9
$N = 4096,\ d = 128$	272.3–275.1	279.2–285.4	287.3–287.6	295.6–295.9	306.4–307.9

16 was fastest here and also at $N = 1024$ and 2048, so the final code uses it.

Three other changes were tested, all with `IB` $= 64$:

- A 4×16 tile (2 loads + 4 broadcasts per 8 FMAs instead of 1 + 8). It won only at $N = 4096,\ d = 64$ (190.0 vs 199.2 ms) and lost badly at $N = 2048$ (62.6 vs 41.7 ms), so 8×8 stayed.
- Padding `Kt` rows to $N + 16$ floats. Without it, the rows read one after another are $4N$ B apart, a multiple of 4 KiB, and land in the same 8-way L1 set. At $d = 128$ the time went from 338.0 to 283 ms.
- `size_t` instead of `int` for index arithmetic, since an `int` index has to be sign-extended before it can be used in an address. A copy with `int` indices ran 20.34×10^9 instructions in 338.8 ms at $N = 4096,\ d = 64$, against 10.38×10^9 in 188.1 ms (`perf stat -e instructions:u`).

Input	Time (ms)	GFLOP/s	% of P_1	Speedup	vs. `loops`
$N = 1024,\ d = 64$	8.9	30.04	43.7	×14.91	×1.67
$N = 2048,\ d = 64$	37.4	28.73	41.8	×14.33	×1.59
$N = 4096,\ d = 64$	180.9	23.74	34.5	×12.23	×1.49
$N = 4096,\ d = 128$	272.7	31.50	45.8	×20.27	×1.57

Limitations

The matmuls themselves are in decent shape now: at $N = 1024$ they run at 49 GFLOP/s ($QK^\top$, 2.72 ms) and 57 GFLOP/s (PV, 2.34 ms), 71% and 83% of P_1. The softmax hasn't changed, and

at $N = 4096,\ d = 64$ it takes as long as $QK^\top$: 69.6 ms against 71.6 ms, with PV at 41.6 ms. That makes it 38% of the time.

The other cost is S itself. It is 64 MiB and allocated fresh on every call, so the first write to each 4 KiB page in $QK^\top$ takes a page fault. The fault count matches the page count:

$$\frac{99{,}157 \text{ faults}}{6 \text{ calls}} \approx 16{,}500 \approx \frac{64 \text{ MiB}}{4 \text{ KiB}} = 16{,}384 \text{ pages of } S.$$

(c) `attn_fused.c`

`attn_fused.c`: fuse the three steps row by row so the $N \times N$ matrix is never stored.

Implementation

All three steps are done for a block of `BR` rows before moving to the next block, so only a `BR` $\times N$ buffer of scores exists at any time. The matmuls inside a block reuse the tiles from (b). The handout says "row by row", but a first version that really did one row at a time (`BR` $= 1$) re-read all of `Kt` and V for every row; with a block, each pass is shared by `BR` rows. One run per value (ms):

`BR`	1	8	16	32	64
$N = 2048,\ d = 64$	53.6	48.4	36.0	35.7	35.3
$N = 4096,\ d = 64$	211.5	137.5	152.4	150.6	152.1
$N = 4096,\ d = 128$	726.0	461.8	362.2	285.1	268.9

64 was kept. It is the fastest at $N \le 2048$ and at $d = 128$, though `BR` $= 8$ beats it by 10% at $N = 4096,\ d = 64$. One smaller change: rather than dividing all N scores of a row by its sum, the output row is scaled once after PV, which is d multiplications instead of N divisions:

$$O_i \leftarrow O_i \times \frac{1}{\sum_j e^{s_{ij} - m_i}}.$$

Input	Time (ms)	GFLOP/s	% of P_1	Speedup	vs. `tiled`
$N = 1024,\ d = 64$	8.8	30.49	44.3	×15.14	×1.02
$N = 2048,\ d = 64$	36.9	29.09	42.3	×14.51	×1.01
$N = 4096,\ d = 64$	156.9	27.37	39.8	×14.10	×1.15
$N = 4096,\ d = 128$	271.0	31.70	46.1	×20.40	×1.01

Limitations

Getting rid of S worked: page faults dropped from 99,157 to 3,282, $QK^\top$ from 72 to 48 ms and the softmax from 70 to 49 ms. But the softmax still calls the library `expf` for every score (the `call` is still in `objdump -d build/fused`), and at 49.0 ms it now costs as much as either matmul (47.9 ms $QK^\top$, 44.2 ms PV).

The score block is also bigger than the 512 KiB L2:

$$64 \times N \times 4 \text{ B} = 1 \text{ MiB} \quad (N = 4096).$$

and L2 misses are still 114.4×10^6 (counter table), not far from `tiled`.

(d) `attn_best.c`

`attn_best.c`: anything you want. Get the maximum single-threaded speedup you can, and explain what you did and why it works.

Implementation

Vectorised exponential. Since clang cannot vectorise a call to `expf`, the exponential is computed inline by `fast_exp`:

$$e^x = 2^n e^r, \qquad n = \lfloor x \log_2 e + 0.5 \rfloor, \qquad r = x - n \ln 2$$

e^r is a degree-6 polynomial (the Cephes `expf` coefficients) and 2^n is written straight into the exponent bits. The row max and sum use 8 partial values each, so neither is one long dependent chain. `objdump -d build/best` has no `expf` call left. On `fused` alone this cut the softmax at $N = 4096$, $d = 64$ from 49.0 to 26.0 ms.

Online softmax (Milakov and Gimelshein, 2018, as in FlashAttention), to make the score block fit in L2. A row's max and sum are normally known only after all N keys; instead, a running max m, sum ℓ and output o are kept per row and rescaled when the max grows, so each 64-row block can go through the keys `BC` columns at a time:

$$m' = \max\bigl(m, \max_j s_j\bigr), \qquad \alpha = e^{m-m'},$$

$$\ell \leftarrow \alpha\ell + \sum_j e^{s_j - m'}, \qquad o \leftarrow \alpha o + \sum_j e^{s_j - m'} v_j, \qquad O = \frac{o}{\ell} \text{ at the end.}$$

`BC` was picked from 2 runs per value (ms; "none" means `fused` with `fast_exp` but no online softmax):

`BC`	none	128	256	512	1024
$N = 4096,\ d = 64$	113.6–115.6	96.9–97.8	94.8–96.5	96.5–105.0	105.6–111.9
$N = 4096,\ d = 128$	235.7–238.8	184.5–187.6	182.4–192.5	186.7–194.7	211.5–221.8

256 was kept (also within noise of the best at $N = 1024$ and 2048). Its tile fits in L2, and L2 misses fall from 114.4×10^6 to 3.7×10^6 (counter table):

$$64 \times 256 \times 4 \text{ B} = 64 \text{ KiB}.$$

Input	Time (ms)	GFLOP/s	% of P_1	Speedup	vs. `fused`
$N = 1024,\ d = 64$	6.1	43.81	63.7	×21.75	×1.44
$N = 2048,\ d = 64$	24.6	43.65	63.4	×21.77	×1.50
$N = 4096,\ d = 64$	99.3	43.25	62.9	×22.27	×1.58
$N = 4096,\ d = 128$	185.0	46.42	67.5	×29.88	×1.46

Limitations

What remains is the matmul code. For one run (6 calls) at $N = 4096$, $d = 64$, `perf stat -e cycles:u` counts 2.43×10^9 cycles:

$$\frac{25.8 \times 10^9 \text{ FLOP}}{2.43 \times 10^9 \text{ cycles}} = 10.6 \text{ FLOP/cycle} \quad \text{(peak 16)}.$$

Other tile shapes (4×16, 2×32, 8×16) were within 5% of 8×8. An earlier AVX2-intrinsics version of `tiled` was 5–13% faster than the plain-C one, so part of the gap is probably how clang schedules the plain-C tile.

Q4. OpenMP **(20 marks)**

Speed up your best variant with OpenMP in `attn_omp.c`. Run it with different thread counts and report the time, speedup and parallel efficiency. Explain the scaling you see.

Implementation

`attn_omp.c` is `attn_best.c` with two OpenMP directives added. Each thread allocates its own score tile:

```
#pragma omp parallel for
for(int j = 0 ; j < N ; j++){
    for(int k = 0 ; k < d ; k++){
        Kt[(size_t)k*(N+16) + j] = K[(size_t)j*d + k] ;
    }
}

#pragma omp parallel
{
    float *S = malloc((size_t)BR * BC * sizeof(float)) ;
    #pragma omp for schedule(dynamic)
    for(int i0 = 0 ; i0 < N ; i0 += BR){
        /* row-block body, unchanged from attn_best.c */
    }
    free(S) ;
}
```

Each block writes only its own rows of O, so the threads never need to synchronise.

Time, speedup and efficiency

$$S_p = \frac{t_1}{t_p}, \qquad E_p = \frac{S_p}{p} \times 100\%$$

where t_p is the time with p threads (warm-up, then median of 3 runs of the command below).

```
$ OMP_NUM_THREADS=p ./build/omp input/<file> /dev/null
```

	$N = 4096,\ d = 64$			$N = 4096,\ d = 128$		
p	t_p (ms)	S_p	E_p	t_p (ms)	S_p	E_p
1	97.0	1.00	100%	200.1	1.00	100%
2	51.8	1.87	94%	97.2	2.06	103%
3	36.8	2.63	88%	72.3	2.77	92%
4	27.5	3.53	88%	53.1	3.77	94%
6	27.5	3.53	59%	50.6	3.95	66%
8	27.3	3.56	44%	50.9	3.93	49%

The smaller inputs scale about the same: $S_4 = 3.93$ and 3.94, $S_8 = 4.10$ and 3.85 at $N = 1024$ and 2048 (all rows are in `results.csv`). With 8 threads the kernel reaches 158–182 GFLOP/s, 60–70% of P_4.

Scaling

Up to 4 threads, one per core, scaling is close to linear because the row blocks are independent. Part of the gap to $\times 4$ is the clock, which drops from 4.18–4.24 GHz with one busy core to 4.03–4.16 GHz with four (`scaling_cur_freq`, as in Q1). The dip at 3 threads (88–94%) is unexplained; pinning the threads to separate cores barely changed it (72.4–72.6 vs 74.3 ms).

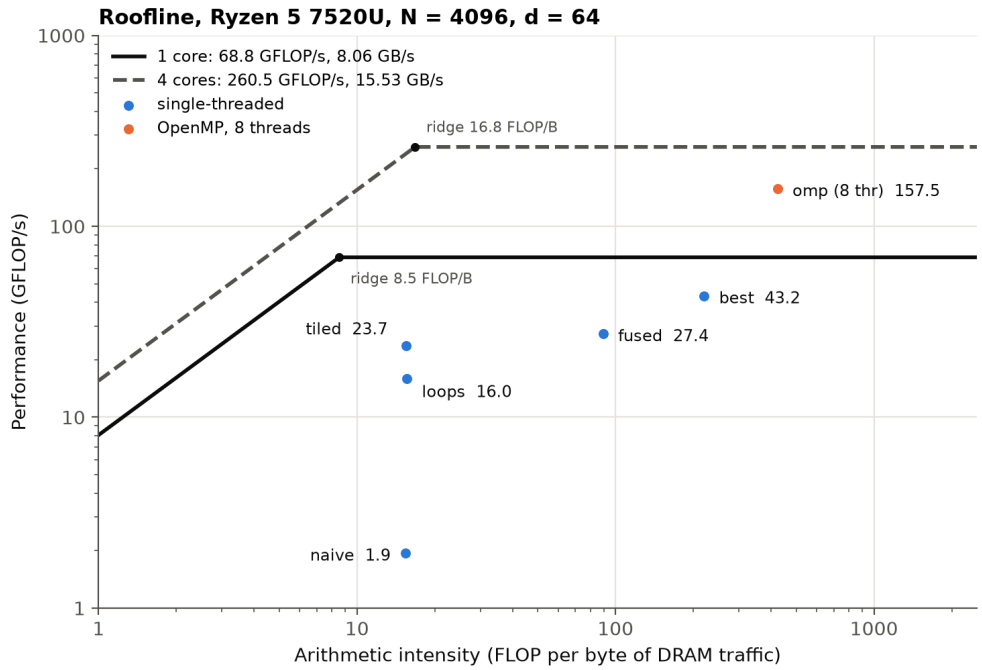
Past 4 threads there is almost nothing to gain: threads 5–8 are SMT siblings on cores that are already busy, sharing their FMA unit. 4 threads on only 2 cores take 45.8–46.7 ms at $N = 4096$, $d = 64$, against 25.3–25.7 ms on 4 cores. Memory isn't the limit either; 8 threads use 0.37 of 15.5 GB/s (Q5).

```
$ OMP_NUM_THREADS=4 OMP_PLACES=threads OMP_PROC_BIND=close ./build/omp input/
    qkv_N4096_d64.bin /dev/null   # and spread
```

This is also why the loop uses `schedule(dynamic)`: a static split gives the threads that share a core as many blocks as the others, and everyone waits for them. With 6 threads, static took 31.7–32.7 ms and dynamic 24.9–26.5 ms (at 4 and 8 threads they were within 1.5 ms).

Q5. Roofline **(20 marks)**

Plot the roofline of your machine using your Q1 numbers, and place every variant on it at $N = 4096$, $d = 64$. Explain how you computed each variant's arithmetic intensity and what limits each variant.



Roofline

$$P(I) = \min(P_{\text{peak}},\ B \cdot I)$$

where I is the arithmetic intensity, with $P_1 = 68.8$, $B_1 = 8.06$ for one core and $P_4 = 260.5$, $B_4 = 15.5$ for `omp` (Q1). Each variant is plotted at its measured GFLOP/s from `results.csv`.

Arithmetic intensity

$$I = \frac{4N^2 d}{B_r + B_w}, \qquad B_r = \frac{64 \times L}{6}$$

where B_r, B_w are DRAM bytes read and written per call and L the lines filled from DRAM in a 6-call run.

- L is the sum of three per-process counters: demand fills from DRAM, L1 prefetch fills from DRAM, and L2 prefetches that miss L3 (mean of 2 runs). The system-wide DRAM counters weren't usable on this machine.

  ```
  $ perf stat -e ls_refills_from_sys.ls_mabresp_lcl_dram:u,ls_hw_pf_dc_fill.
      ls_mabresp_lcl_dram:u,l2_pf_miss_l2_l3:u ./build/<variant> input/qkv_N4096_d64.
      bin /dev/null
  ```

- B_w has no counter, so it is estimated from what each variant writes: S (64 MiB) twice in `naive`, `loops` and `tiled`, O (1 MiB) in all, and `Kt` (1 MiB) in all but `naive`.

At $N = 4096$, $d = 64$: $4N^2d = 4.295 \times 10^9$ FLOP, $|S| = 4N^2\,\mathrm{B} = 67.11\,\mathrm{MB}$ and $|O| = |\texttt{Kt}| = 4Nd\,\mathrm{B} = 1.05\,\mathrm{MB}$; L is in millions of lines.

$$\texttt{naive}: \quad B_r = \frac{64 \times 13.520}{6} = 144.22, \quad B_w = 2(67.11) + 1.05 = 135.27, \qquad I = \frac{4295}{279.49} = 15.4$$

$$\texttt{loops}: \quad B_r = \frac{64 \times 13.070}{6} = 139.41, \quad B_w = 2(67.11) + 2(1.05) = 136.31, \quad I = \frac{4295}{275.72} = 15.6$$

$$\texttt{tiled}: \quad B_r = \frac{64 \times 13.272}{6} = 141.56, \quad B_w = 2(67.11) + 2(1.05) = 136.31, \quad I = \frac{4295}{277.87} = 15.5$$

$$\texttt{fused}: \quad B_r = \frac{64 \times 4.297}{6} = 45.83, \qquad B_w = 2(1.05) = 2.10, \qquad I = \frac{4295}{47.93} = 89.6$$

$$\texttt{best}: \quad B_r = \frac{64 \times 1.631}{6} = 17.40, \qquad B_w = 2(1.05) = 2.10, \qquad I = \frac{4295}{19.50} = 220.3$$

$$\texttt{omp}: \quad B_r = \frac{64 \times 0.754}{6} = 8.04, \qquad B_w = 2(1.05) = 2.10, \qquad I = \frac{4295}{10.14} = 423.5$$

with B_r, B_w and the denominator $B_r + B_w$ in MB, and I in FLOP/B.

Variant	I (FLOP/B)	GFLOP/s	% of peak
`naive`	15.4	1.94	2.8
`loops`	15.6	15.96	23.2
`tiled`	15.5	23.74	34.5
`fused`	89.6	27.37	39.8
`best`	220.3	43.25	62.9
`omp` (8 threads)	423.5	157.54	60.5

These counts can move by about 30% between runs (a later re-run gave 188.8 MB for `naive` and 20.4 MB for `best`). As a sanity check, counting what `naive` must read gives 204 MB, against 144–189 MB measured.

Limitations

None of the variants is limited by DRAM bandwidth. They all sit right of their ridge point ($I \geq 15.4$ against 8.5, and 423.5 against 16.8 for `omp`), and the most bandwidth any of them uses is

$$\frac{B_r + B_w}{t} = \frac{277.9\ \mathrm{MB}}{180.9\ \mathrm{ms}} = 1.5\ \mathrm{GB/s} \quad (\texttt{tiled}),$$

out of 8.06 GB/s. So the distance below the roof comes from inside the core, and it is a different thing for each variant:

- `naive`: scalar FMAs that each wait for the previous one (Q2).
- `loops`: running sums in memory, and a scalar softmax (Q3a).
- `tiled`: the scalar softmax, and page faults on the 64 MiB S (Q3b).
- `fused`: no more S, which lifts I from 15.5 to 89.6, but still the scalar `expf` and a score block larger than L2 (Q3c).
- `best`: the plain-C matmul tiles (Q3d).
- `omp`: the same per-core limit as `best`; the SMT threads add almost nothing (Q4). It reads less than `best`, probably because threads running at the same time share the `Kt` and V tiles in L3.